

StreamFlix Video Streaming Service Management System

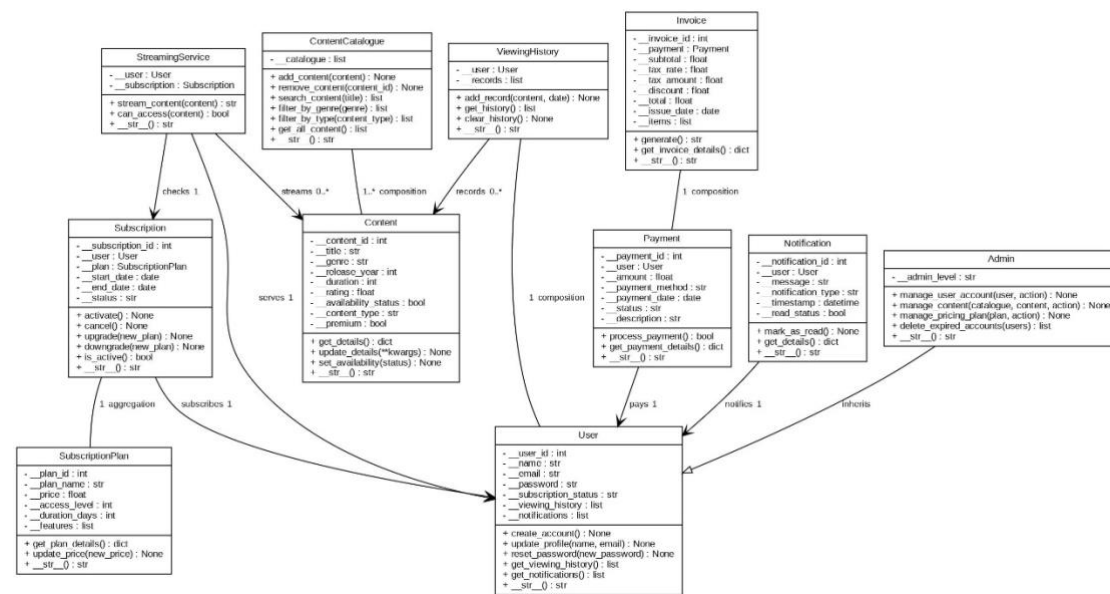
Complete Assignment Solution

StreamFlix Video Streaming Service Management System

ICS220 — Programming Fundamentals | Assignment 2
Student Name:

ID :

Section A: UML Class Diagram and Description



Description:

The system comprises 11 classes that model the StreamFlix domain. The central entity is the **User** class, which stores personal details (name, email, password), subscription status, viewing history, and notifications. The **Admin** class inherits from **User** (IS-A relationship) and adds elevated privileges for managing user accounts, content, and pricing plans.

Content represents a single video item (Movie, TV Series, or Documentary) with attributes for title, genre, release year, duration, rating, availability, and premium status. **ContentCatalogue** manages a collection of **Content** objects through a composition relationship — the catalogue owns its content items and removing them from the catalogue deletes them from the system (multiplicity 1 to many).

SubscriptionPlan defines subscription tiers (Basic, Standard, Premium) with pricing, access levels, duration, and features. Plans exist independently and can be shared across subscriptions. **Subscription** links a **User** to a **SubscriptionPlan** with start/end dates and status tracking. The relationship between **Subscription** and **SubscriptionPlan** is aggregation (the plan exists independently), while the relationship with **User** is a standard association.

Payment records financial transactions for a user. **Invoice** is generated from a completed **Payment** through a composition relationship — an invoice cannot exist without its parent payment. It calculates subtotal, tax, discount, and total.

ViewingHistory tracks content viewing records through a composition relationship with **User** (the history is owned by the user) and an association with **Content**. **Notification** represents messages sent to users. **StreamingService** controls content access by checking subscription eligibility and content availability.

Relationships: Inheritance (Admin extends User), Composition (ContentCatalogue–Content, Invoice–Payment, ViewingHistory–User), Aggregation (Subscription–SubscriptionPlan), Associations (Payment–User, Notification–User, StreamingService–User/Subscription/Content).

Assumptions: Content rating uses a 0.0–10.0 scale. Three subscription tiers with access levels 1, 2, 3. Premium content requires level 3. Five payment methods supported. Tax rate defaults to 5%. Four notification types. Auto-incrementing ID counters for uniqueness.

Section B: Python Classes

File 5: viewing_history.py

"""
viewing_history.py

Defines the ViewingHistory class that records and manages
a user's content viewing activity on StreamFlix.
Author: [Your Name]
Date: [Submission Date]
"""
from datetime import date
class ViewingHistory:
"""
Tracks the viewing history for a specific user.
Uses composition – the history belongs to and is owned by the
user.
Attributes:
__user (User): The user whose history is tracked.
__records (list[dict]): List of records with 'content' and
'date_watched'.
"""
def __init__(self, user):
"""
Initialise a ViewingHistory for a given user.
Args:
user (User): The user to track.
"""
self.__user = user
self.__records = []
def get_user(self):
"""Return the user associated with this history."""
return self.__user
def add_record(self, content, date_watched: date = None) ->
None:
"""
Add a viewing record.

Args:
content (Content): The content that was watched.
date_watched (date, optional): Date of viewing.
Defaults to today.
Raises:
TypeError: If content is not a Content instance.
"""
if date_watched is None:
date_watched = date.today()
record = {
'content_id': content.get_content_id(),
'title': content.get_title(),
'content_type': content.get_content_type(),
'date_watched': date_watched
}
self.__records.append(record)
Also add to the user's own history reference
self.__user.add_viewing_record(record)
def get_history(self) -> list:
"""
Return a copy of all viewing records.
Returns:
list[dict]: List of viewing records.
"""
return list(self.__records)
def clear_history(self) -> None:
"""Clear all viewing records for this user."""
self.__records.clear()
print(f"Viewing history cleared for
{self.__user.get_name()}").
def __str__(self) -> str:
"""Return a formatted viewing history."""
if not self.__records:
return f"Viewing History for {self.__user.get_name()}:
[No records]"
lines = [
f"{'=' * 55}",
f" Viewing History for {self.__user.get_name()}",
f"{'=' * 55}"
]
for i, rec in enumerate(self.__records, 1):
lines.append(
f" {i}. {rec['title']} ({rec['content_type']}) "
f"- Watched: {rec['date_watched']}"
)
lines.append(f"{'=' * 55}")
lines.append(f" Total watched: {len(self.__records)}")
return "\n".join(lines)
File 6: notification.py
"""

notification.py

Defines the Notification class for sending and managing user notifications on the StreamFlix platform.
Author: [Your Name]
Date: [Submission Date]
"""
from datetime import datetime
class Notification:
"""
Represents a notification sent to a user.
Attributes:
__notification_id (int): Unique notification identifier.
__user (User): The recipient user.
__message (str): Notification message.
__notification_type (str): Type - 'Payment', 'Subscription', 'Content', 'General'.
__timestamp (datetime): When the notification was created.
__read_status (bool): Whether the user has read it.
"""
_id_counter = 7000
VALID_TYPES = ['Payment', 'Subscription', 'Content', 'General']
def __init__(self, user, message: str, notification_type: str = "General"):
"""
Create a new Notification.
Args:
user (User): The recipient.
message (str): The notification message.
notification_type (str): One of the valid types.
Default 'General'.
Raises:
ValueError: If notification_type is invalid or message is empty.
"""
if notification_type not in Notification.VALID_TYPES:
raise ValueError(f"Type must be one of {Notification.VALID_TYPES}.")
if not message.strip():
raise ValueError("Notification message cannot be empty.")
Notification._id_counter += 1
self.__notification_id = Notification._id_counter
self.__user = user
self.__message = message.strip()
self.__notification_type = notification_type
self.__timestamp = datetime.now()
self.__read_status = False
Automatically add to user's notification list

user.add_notification(self)
— Getters —————
def get_notification_id(self) -> int:
"""Return the notification ID."""
return self.__notification_id
def get_message(self) -> str:
"""Return the notification message."""
return self.__message
def get_type(self) -> str:
"""Return the notification type."""
return self.__notification_type
def is_read(self) -> bool:
"""Return True if the notification has been read."""
return self.__read_status
def mark_as_read(self) -> None:
"""Mark the notification as read."""
self.__read_status = True
def get_details(self) -> dict:
"""
Return all notification details as a dictionary.
Returns:
dict: Notification attributes.
"""
return {
'notification_id': self.__notification_id,
'user': self.__user.get_name(),
'message': self.__message,
'type': self.__notification_type,
'timestamp': str(self.__timestamp),
'read': self.__read_status
}
def __str__(self) -> str:
"""Return a formatted notification string."""
read_marker = "Read" if self.__read_status else "Unread"
return (f"[{self.__notification_type}] ({read_marker}) "
f"To: {self.__user.get_name()} {self.__message} "
f" {self.__timestamp.strftime('%Y-%m-%d %H:%M')}")

File 7: streaming_service.py

"""
streaming_service.py

Defines the StreamingService class that controls content access based on user subscription and content availability.
Author: [Your Name]
Date: [Submission Date]
"""

class StreamingService:
"""
Manages the streaming logic – checks subscription eligibility
and content availability before granting access.
Attributes:
__user (User): The user requesting the stream.
__subscription (Subscription): The user's active
subscription.
"""
def __init__(self, user, subscription):
"""
Initialise the StreamingService for a user/subscription
pair.
Args:
user (User): The streaming user.
subscription (Subscription): The user's subscription.
"""
self.__user = user
self.__subscription = subscription
def can_access(self, content) -> bool:
"""
Determine whether the user can access the given content.
Access rules:
- Subscription must be active.
- Content must be available.
- Non-premium content: accessible by any active
subscription.
- Premium content: requires access level >= 3 (Premium
plan).
Args:
content (Content): The content to check.
Returns:
bool: True if the user can access the content.
"""
if not self.__subscription.is_active():
return False
if not content.is_available():
return False
if content.is_premium():
return
self.__subscription.get_plan().get_access_level() >= 3
return True
def stream_content(self, content) -> str:
"""
Attempt to stream the content for the user.
Args:
content (Content): The content to stream.

Returns:
str: A message indicating success or the reason for denial.
"""
if not self.__subscription.is_active():
return (f"Access Denied: {self.__user.get_name()} 's subscription "
f"is not active. Please renew your subscription.")
if not content.is_available():
return (f"Access Denied: '{content.get_title()}' is currently "
f"unavailable for streaming.")
if content.is_premium() and self.__subscription.get_plan().get_access_level() < 3:
return (f"Access Denied: '{content.get_title()}' is premium content. "
f"Please upgrade to Premium plan to access this content.")
return (f"Now streaming '{content.get_title()}' for "
f"{self.__user.get_name()}. Enjoy watching!")
def __str__(self) -> str:
"""Return a string representation."""
return (f"StreamingService User: {self.__user.get_name()} "
f"Plan: {self.__subscription.get_plan().get_plan_name()} "
f"Active: {self.__subscription.is_active()}")

File 8: main.py (Demonstration / Driver)

"""
main.py

Main driver file that demonstrates the full functionality of the StreamFlix Video Streaming Service Management System.
Author: [Your Name]
Date: [Submission Date]
"""
from content import Content, ContentCatalogue
from user import User, Admin
from subscription import SubscriptionPlan, Subscription
from payment import Payment, Invoice
from viewing_history import ViewingHistory
from notification import Notification
from streaming_service import StreamingService
from datetime import date
def print_section(title: str) -> None:
"""Print a formatted section header."""
print(f"\n{'#' * 60}")
print(f"# {title}")
print(f"{'#' * 60}\n")

```

def main():
    """Main demonstration function for StreamFlix system."""

    # =====
    # 1. CREATE SUBSCRIPTION PLANS
    # =====
    print_section("1. SUBSCRIPTION PLANS")

    basic_plan = SubscriptionPlan(
        plan_name="Basic", price=9.99, access_level=1,
        duration_days=30, features=["SD Streaming", "1 Device",
        "Limited Library"]
    )
    standard_plan = SubscriptionPlan(
        plan_name="Standard", price=14.99, access_level=2,
        duration_days=30, features=["HD Streaming", "2 Devices",
        "Full Library"]
    )
    premium_plan = SubscriptionPlan(
        plan_name="Premium", price=19.99, access_level=3,
        duration_days=30, features=["4K Streaming", "4 Devices",
        "Full Library", "Premium Content"]
    )

    print(basic_plan)
    print(standard_plan)
    print(premium_plan)

    # =====
    # 2. CREATE USERS AND ADMIN
    # =====
    print_section("2. USER & ADMIN CREATION")

    user1 = User("Alice Johnson", "alice@email.com", "Pass123")
    user1.create_account()
    print(user1)

    user2 = User("Bob Smith", "bob@email.com", "Secure456")
    user2.create_account()
    print(user2)

    admin = Admin("Sarah Admin", "admin@streamflix.com",
    "Admin789", "Super")
    admin.create_account()
    print(admin)

    # =====
    # 3. CREATE CONTENT CATALOGUE
    # =====
    print_section("3. CONTENT CATALOGUE")

    catalogue = ContentCatalogue()

    movie1 = Content("The Matrix", "Sci-Fi", 1999, 136, 8.7,
    "Movie")
    movie2 = Content("Inception", "Sci-Fi", 2010, 148, 8.8,
    "Movie", premium=True)

```

```

    series1 = Content("Breaking Bad", "Drama", 2008, 49, 9.5, "TV
Series")
    series2 = Content("Stranger Things", "Sci-Fi", 2016, 51, 8.7,
"TV Series")
    doc1 = Content("Planet Earth", "Nature", 2006, 50, 9.4,
"Documentary")
    doc2 = Content("The Social Dilemma", "Technology", 2020, 94,
7.6, "Documentary", premium=True)

    for c in [movie1, movie2, series1, series2, doc1, doc2]:
        catalogue.add_content(c)

    print(catalogue)

# =====
# 4. SUBSCRIBE USERS
# =====
print_section("4. SUBSCRIPTIONS")

sub1 = Subscription(user1, basic_plan)
print(sub1)
print(f"    Is active? {sub1.is_active()}")

sub2 = Subscription(user2, premium_plan)
print(sub2)
print(f"    Is active? {sub2.is_active()}")

# =====
# 5. UPGRADE / DOWNGRADE
# =====
print_section("5. SUBSCRIPTION UPGRADE / DOWNGRADE")

sub1.upgrade(standard_plan)
print(sub1)

sub2.downgrade(standard_plan)
print(sub2)

# Re-upgrade user1 to Premium for streaming tests
sub1.upgrade(premium_plan)
print(sub1)

# =====
# 6. STREAMING CONTENT
# =====
print_section("6. STREAMING CONTENT")

stream1 = StreamingService(user1, sub1)
print(stream1.stream_content(movie1))    # Non-premium – should
work
print(stream1.stream_content(movie2))    # Premium – user1 now
Premium
print(stream1.stream_content(doc2))      # Premium – should work

stream2 = StreamingService(user2, sub2)
print(stream2.stream_content(series1))   # Non-premium – should
work
print(stream2.stream_content(movie2))    # Premium – user2 is
Standard now

```

Test unavailable content
doc1.set_availability(False)
print(stream1.stream_content(doc1)) # Unavailable
=====
7. PAYMENT PROCESSING
=====
print_section("7. PAYMENT PROCESSING")
payment1 = Payment(user1, 19.99, "Credit Card", "Premium Subscription - Monthly")
print(f"Before processing: {payment1}")
payment1.process_payment()
print(f"After processing: {payment1}")
payment2 = Payment(user2, 14.99, "PayPal", "Standard Subscription - Monthly")
payment2.process_payment()
print(payment2)
=====
8. INVOICE GENERATION
=====
print_section("8. INVOICE GENERATION")
invoice1 = Invoice(payment1, tax_rate=0.05, discount=2.00, items=["Premium Plan - Monthly", "Discount: Returning Customer"])
print(invoice1.generate())
invoice2 = Invoice(payment2, tax_rate=0.05, discount=0.00, items=["Standard Plan - Monthly"])
print(invoice2.generate())
=====
9. VIEWING HISTORY
=====
print_section("9. VIEWING HISTORY")
history1 = ViewingHistory(user1)
history1.add_record(movie1, date(2025, 3, 1))
history1.add_record(movie2, date(2025, 3, 5))
history1.add_record(series1, date(2025, 3, 8))
print(history1)
history2 = ViewingHistory(user2)
history2.add_record(series1, date(2025, 3, 2))
history2.add_record(doc1, date(2025, 3, 4))
print(history2)
=====
10. NOTIFICATIONS
=====
print_section("10. NOTIFICATIONS")
notif1 = Notification(user1, "Your Premium subscription is now active!", "Subscription")

```

    notif2 = Notification(user1, "Payment of $19.99 received. Thank
you!", "Payment")
    notif3 = Notification(user2, "New content available: Stranger
Things S5!", "Content")
    notif4 = Notification(user2, "Your Standard subscription is now
active!", "Subscription")

    print(notif1)
    print(notif2)
    notif1.mark_as_read()
    print(f"After reading: {notif1}")
    print(notif3)
    print(notif4)

    print(f"\n{user1.get_name()}'s notifications:
{len(user1.get_notifications())}")
    print(f"{user2.get_name()}'s notifications:
{len(user2.get_notifications())}")

# =====
# 11. ADMIN OPERATIONS
# =====
print_section("11. ADMIN OPERATIONS")

# Admin manages content
new_movie = Content("Dune: Part Three", "Sci-Fi", 2025, 165,
0.0, "Movie", premium=True)
admin.manage_content(catalogue, new_movie, 'add')
print(f"Catalogue size after adding:
{len(catalogue.get_all_content())}")

# Admin resets user password
admin.manage_user_account(user1, 'reset_password')

# Admin deactivates account
user3 = User("Charlie Expired", "charlie@email.com",
"Test1234")
user3.set_subscription_status("Cancelled")
all_users = [user1, user2, user3]
all_users = admin.delete_expired_accounts(all_users)
print(f"Active users remaining: {len(all_users)}")

# =====
# 12. CONTENT SEARCH AND FILTER
# =====
print_section("12. CONTENT SEARCH & FILTER")

results = catalogue.search_content("the")
print(f"Search 'the': {len(results)} results")
for r in results:
    print(f"    - {r}")

sci-fi = catalogue.filter_by_genre("Sci-Fi")
print(f"\nSci-Fi content: {len(sci-fi)}")
for s in sci-fi:
    print(f"    - {s}")

movies = catalogue.filter_by_type("Movie")
print(f"\nMovies: {len(movies)}")

```

for m in movies:
print(f" - {m}")
=====
13. SUBSCRIPTION CANCELLATION
=====
print_section("13. SUBSCRIPTION CANCELLATION")
sub1.cancel()
print(sub1)
print(f"Is active? {sub1.is_active()}")
Try to stream after cancellation
print(stream1.stream_content(movie1))
print("\n" + "=" * 60)
print(" StreamFlix Demonstration Complete!")
print("=" * 60)
if __name__ == "__main__":
main()

File 9: test_streamflix.py (Test Cases)

"""
test_streamflix.py

Comprehensive test cases for the StreamFlix system.
Tests all features with 2 examples each, including exception
handling.
Author: [Your Name]
Date: [Submission Date]
"""
from content import Content, ContentCatalogue
from user import User, Admin
from subscription import SubscriptionPlan, Subscription
from payment import Payment, Invoice
from viewing_history import ViewingHistory
from notification import Notification
from streaming_service import StreamingService
from datetime import date
def print_test_header(test_name: str, test_number: int) -> None:
"""Print a formatted test header."""
print(f"\n{'—' * 60}")
print(f" TEST {test_number}: {test_name}")
print(f"{'—' * 60}")
def print_result(passed: bool, description: str) -> None:
"""Print a PASS/FAIL result line."""
status = "✓ PASS" if passed else "X FAIL"
print(f" {status}: {description}")

=====
TEST 1: USER ACCOUNT CREATION
=====
def test_user_account_creation():
"""Test user account creation with valid and invalid inputs."""
print_test_header("USER ACCOUNT CREATION", 1)
Example 1: Successful user creation
print("\n --- Example 1: Valid user creation ---")
user1 = User("Alice Johnson", "alice@email.com", "Pass123")
print_result(user1.get_name() == "Alice Johnson", "Name stored correctly")
print_result(user1.get_email() == "alice@email.com", "Email stored correctly")
print_result(user1.get_subscription_status() == "Inactive", "Default status is Inactive")
print(f" {user1}")
Example 2: Another successful user creation
print("\n --- Example 2: Valid user creation with different details ---")
user2 = User("Bob Smith", "bob.smith@company.org", "Secure789")
print_result(user2.get_name() == "Bob Smith", "Name stored correctly")
print_result(user2.get_email() == "bob.smith@company.org", "Email stored correctly")
print_result(user2.get_user_id() != user1.get_user_id(), "Unique IDs assigned")
print(f" {user2}")
Exception handling tests
print("\n --- Exception Handling ---")
try:
bad_user = User("", "test@email.com", "Pass123")
print_result(False, "Empty name should raise ValueError")
except ValueError as e:
print_result(True, f"Empty name rejected: {e}")
try:
bad_user = User("Test User", "invalid-email", "Pass123")
print_result(False, "Invalid email should raise ValueError")
except ValueError as e:
print_result(True, f"Invalid email rejected: {e}")
try:
bad_user = User("Test User", "test@email.com", "123")
print_result(False, "Short password should raise ValueError")
except ValueError as e:
print_result(True, f"Short password rejected: {e}")
Test profile update
print("\n --- Profile Update ---")
user1.update_profile(name="Alice J. Williams", email="alice.w@email.com")

```

    print_result(user1.get_name() == "Alice J. Williams", "Name
updated successfully")
    print_result(user1.get_email() == "alice.w@email.com", "Email
updated successfully")

# =====
# TEST 2: CONTENT CATALOGUE MANAGEMENT
# =====
def test_content_catalogue_management():
    """Test adding, modifying, and removing content in the
catalogue."""
    print_test_header("CONTENT CATALOGUE MANAGEMENT", 2)

    catalogue = ContentCatalogue()

    # Example 1: Add a movie
    print("\n --- Example 1: Add and verify a Movie ---")
    movie = Content("Interstellar", "Sci-Fi", 2014, 169, 8.6,
"Movie")
    catalogue.add_content(movie)
    print_result(len(catalogue.get_all_content()) == 1, "Movie
added to catalogue")
    print_result(movie.get_title() == "Interstellar", "Title
correct")
    print_result(movie.get_genre() == "Sci-Fi", "Genre correct")
    print_result(movie.get_release_year() == 2014, "Release year
correct")
    print(f"    {movie}")

    # Example 2: Add a TV series and documentary
    print("\n --- Example 2: Add TV Series and Documentary ---")
    series = Content("The Crown", "Drama", 2016, 58, 8.6, "TV
Series")
    doc = Content("Our Planet", "Nature", 2019, 50, 9.3,
"Documentary", premium=True)
    catalogue.add_content(series)
    catalogue.add_content(doc)
    print_result(len(catalogue.get_all_content()) == 3, "3 items in
catalogue")
    print_result(doc.is_premium(), "Documentary is marked as
premium")
    print(f"    {series}")
    print(f"    {doc}")

    # Modify content
    print("\n --- Modify Content ---")
    movie.update_details(rating=9.0, genre="Sci-Fi/Adventure")
    print_result(movie.get_rating() == 9.0, "Rating updated to
9.0")
    print_result(movie.get_genre() == "Sci-Fi/Adventure", "Genre
updated")

    # Remove content
    print("\n --- Remove Content ---")
    content_id = series.get_content_id()
    catalogue.remove_content(content_id)
    print_result(len(catalogue.get_all_content()) == 2, "Item
removed, 2 remaining")

```

Exception handling
print("\n --- Exception Handling ---")
try:
bad_content = Content("Bad Movie", "Action", 2023, 120, 11.0, "Movie")
print_result(False, "Rating > 10 should raise ValueError")
except ValueError as e:
print_result(True, f"Invalid rating rejected: {e}")
try:
bad_content = Content("Bad Type", "Drama", 2023, 120, 7.0, "Anime")
print_result(False, "Invalid content_type should raise ValueError")
except ValueError as e:
print_result(True, f"Invalid type rejected: {e}")
try:
catalogue.remove_content(99999)
print_result(False, "Removing non-existent ID should raise ValueError")
except ValueError as e:
print_result(True, f"Non-existent ID rejected: {e}")
=====
TEST 3: SUBSCRIPTION PLAN MANAGEMENT
=====
def test_subscription_plan_management():
"""Test subscription creation, upgrading, downgrading, and cancellation."""
print_test_header("SUBSCRIPTION PLAN MANAGEMENT", 3)
basic = SubscriptionPlan("Basic", 9.99, 1, 30, ["SD", "1 Device"])
standard = SubscriptionPlan("Standard", 14.99, 2, 30, ["HD", "2 Devices"])
premium = SubscriptionPlan("Premium", 19.99, 3, 30, ["4K", "4 Devices", "Premium Content"])
Example 1: Subscribe and upgrade
print("\n --- Example 1: Subscribe to Basic, then Upgrade ---")
user1 = User("TestUser1", "test1@email.com", "Pass123")
sub1 = Subscription(user1, basic)
print_result(sub1.is_active(), "Subscription is active")
print_result(user1.get_subscription_status() == "Active", "User status set to Active")
print(f" Before upgrade: {sub1}")
sub1.upgrade(standard)
print_result(sub1.get_plan().get_plan_name() == "Standard", "Upgraded to Standard")
print(f" After upgrade: {sub1}")
sub1.upgrade(premium)
print_result(sub1.get_plan().get_plan_name() == "Premium", "Upgraded to Premium")

<pre> # Example 2: Subscribe to Premium, then Downgrade, then Cancel print("\n --- Example 2: Subscribe to Premium, Downgrade, Cancel ---") user2 = User("TestUser2", "test2@email.com", "Pass456") sub2 = Subscription(user2, premium) print(f" Initial: {sub2}") sub2.downgrade(standard) print_result(sub2.get_plan().get_plan_name() == "Standard", "Downgraded to Standard") print(f" Downgraded: {sub2}") sub2.cancel() print_result(sub2.get_status() == "Cancelled", "Subscription cancelled") print_result(not sub2.is_active(), "is_active returns False after cancel") print_result(user2.get_subscription_status() == "Cancelled", "User status Cancelled") # Exception handling print("\n --- Exception Handling ---") try: sub1.upgrade(basic) # sub1 is Premium, can't "upgrade" to Basic print_result(False, "Upgrading to lower plan should raise ValueError") except ValueError as e: print_result(True, f"Invalid upgrade rejected: {e}") try: sub_temp = Subscription(User("Temp", "t@e.com", "Pass12"), basic) sub_temp.downgrade(standard) # Can't downgrade from Basic to Standard print_result(False, "Downgrading to higher plan should raise ValueError") except ValueError as e: print_result(True, f"Invalid downgrade rejected: {e}") try: bad_plan = SubscriptionPlan("Bad", -5.0, 1) print_result(False, "Negative price should raise ValueError") except ValueError as e: print_result(True, f"Negative price rejected: {e}") # ===== # TEST 4: STREAMING CONTENT ACCESS # ===== def test_streaming_content_access(): """Test user access to content based on subscription and availability.""" print_test_header("STREAMING CONTENT ACCESS", 4) basic = SubscriptionPlan("Basic", 9.99, 1) premium = SubscriptionPlan("Premium", 19.99, 3) </pre>

regular_content = Content("Free Movie", "Action", 2020, 120, 7.5, "Movie")
premium_content = Content("Exclusive Film", "Drama", 2024, 140, 9.0, "Movie", premium=True)
unavailable = Content("Old Show", "Comedy", 2005, 30, 6.0, "TV Series")
unavailable.set_availability(False)
Example 1: Basic user tries to stream
print("\n --- Example 1: Basic subscriber access ---")
user1 = User("BasicUser", "basic@email.com", "Pass123")
sub1 = Subscription(user1, basic)
stream1 = StreamingService(user1, sub1)
result = stream1.stream_content(regular_content)
print_result("Now streaming" in result, "Basic user can stream regular content")
print(f" {result}")
result = stream1.stream_content(premium_content)
print_result("Access Denied" in result, "Basic user denied premium content")
print(f" {result}")
result = stream1.stream_content(unavailable)
print_result("unavailable" in result.lower(), "Unavailable content denied")
print(f" {result}")
Example 2: Premium user access
print("\n --- Example 2: Premium subscriber access ---")
user2 = User("PremiumUser", "prem@email.com", "Pass456")
sub2 = Subscription(user2, premium)
stream2 = StreamingService(user2, sub2)
result = stream2.stream_content(regular_content)
print_result("Now streaming" in result, "Premium user can stream regular content")
print(f" {result}")
result = stream2.stream_content(premium_content)
print_result("Now streaming" in result, "Premium user can stream premium content")
print(f" {result}")
Test with cancelled subscription
print("\n --- Cancelled subscription test ---")
sub2.cancel()
result = stream2.stream_content(regular_content)
print_result("not active" in result.lower(), "Cancelled user denied streaming")
print(f" {result}")
=====
TEST 5: PAYMENT PROCESSING
=====
def test_payment_processing():

"""Test payment creation and processing with different methods."""
print_test_header("PAYMENT PROCESSING", 5)
user = User("PayUser", "pay@email.com", "Pass123")
Example 1: Credit Card payment
print("\n --- Example 1: Credit Card payment ---")
pay1 = Payment(user, 19.99, "Credit Card", "Premium Subscription")
print_result(pay1.get_status() == "Pending", "Initial status is Pending")
success = pay1.process_payment()
print_result(success, "Payment processed successfully")
print_result(pay1.get_status() == "Completed", "Status updated to Completed")
print(f" {pay1}")
Example 2: PayPal payment
print("\n --- Example 2: PayPal payment ---")
pay2 = Payment(user, 4.99, "PayPal", "Movie Rental: Inception")
success = pay2.process_payment()
print_result(success, "PayPal payment processed")
details = pay2.get_payment_details()
print_result(details['payment_method'] == "PayPal", "Method recorded as PayPal")
print_result(details['amount'] == 4.99, "Amount recorded correctly")
print(f" {pay2}")
Exception handling
print("\n --- Exception Handling ---")
try:
bad_pay = Payment(user, -10.0, "Credit Card", "Bad payment")
print_result(False, "Negative amount should raise ValueError")
except ValueError as e:
print_result(True, f"Negative amount rejected: {e}")
try:
bad_pay = Payment(user, 10.0, "Bitcoin", "Crypto payment")
print_result(False, "Invalid payment method should raise ValueError")
except ValueError as e:
print_result(True, f"Invalid method rejected: {e}")
=====
TEST 6: INVOICE GENERATION
=====
def test_invoice_generation():
"""Test invoice creation with taxes and discounts."""
print_test_header("INVOICE GENERATION", 6)
user = User("InvoiceUser", "inv@email.com", "Pass123")
Example 1: Invoice with tax and discount

print("\n --- Example 1: Subscription invoice with tax & discount ---")
pay1 = Payment(user, 19.99, "Credit Card", "Premium Plan")
pay1.process_payment()
inv1 = Invoice(pay1, tax_rate=0.05, discount=3.00,
items=["Premium Plan - Monthly", "Loyalty Discount"])
details = inv1.get_invoice_details()
print_result(details['subtotal'] == 19.99, f"Subtotal: \${details['subtotal']}")
print_result(details['tax_amount'] == round(19.99 * 0.05, 2),
f"Tax (5%): \${details['tax_amount']}")
print_result(details['discount'] == 3.00, f"Discount: \${details['discount']}")
expected_total = round(19.99 + (19.99 * 0.05) - 3.00, 2)
print_result(details['total'] == expected_total, f"Total: \${details['total']}")
print(inv1.generate())
Example 2: Simple invoice without discount
print("\n --- Example 2: Content purchase invoice (no discount) ---")
pay2 = Payment(user, 4.99, "Debit Card", "Rent: Inception")
pay2.process_payment()
inv2 = Invoice(pay2, tax_rate=0.10, discount=0.0, items=["Movie Rental: Inception"])
details2 = inv2.get_invoice_details()
expected_total2 = round(4.99 + (4.99 * 0.10), 2)
print_result(details2['total'] == expected_total2, f"Total: \${details2['total']}")
print(inv2.generate())
Exception handling
print("\n --- Exception Handling ---")
pending_pay = Payment(user, 9.99, "PayPal", "Pending test")
try:
bad_inv = Invoice(pending_pay)
print_result(False, "Invoice from pending payment should raise ValueError")
except ValueError as e:
print_result(True, f"Pending payment rejected: {e}")
try:
pay_temp = Payment(user, 5.0, "Credit Card", "Tax test")
pay_temp.process_payment()
bad_inv = Invoice(pay_temp, tax_rate=1.5)
print_result(False, "Tax rate > 1.0 should raise ValueError")
except ValueError as e:
print_result(True, f"Invalid tax rate rejected: {e}")
=====
TEST 7: VIEWING HISTORY TRACKING
=====
def test_viewing_history_tracking():
"""Test recording and retrieving viewing history."""
print_test_header("VIEWING HISTORY TRACKING", 7)

user = User("HistoryUser", "hist@email.com", "Pass123")
movie = Content("Interstellar", "Sci-Fi", 2014, 169, 8.6, "Movie")
series = Content("Dark", "Thriller", 2017, 60, 8.8, "TV Series")
doc = Content("Cosmos", "Science", 2014, 44, 9.3, "Documentary")
Example 1: Add records and retrieve history
print("\n --- Example 1: Record and display viewing history ---")
history = ViewingHistory(user)
history.add_record(movie, date(2025, 2, 10))
history.add_record(series, date(2025, 2, 15))
records = history.get_history()
print_result(len(records) == 2, "2 records stored")
print_result(records[0]['title'] == "Interstellar", "First record is Interstellar")
print_result(records[1]['date_watched'] == date(2025, 2, 15), "Date stored correctly")
print(history)
Example 2: Add more records and clear history
print("\n --- Example 2: Add record, then clear history ---")
history.add_record(doc, date(2025, 3, 1))
print_result(len(history.get_history()) == 3, "3 records before clear")
print(history)
history.clear_history()
print_result(len(history.get_history()) == 0, "History cleared successfully")
print(history)
=====
TEST 8: NOTIFICATION SYSTEM
=====
def test_notification_system():
"""Test notification creation, delivery, and read status."""
print_test_header("NOTIFICATION SYSTEM", 8)
user = User("NotifUser", "notif@email.com", "Pass123")
Example 1: Payment notification
print("\n --- Example 1: Payment confirmation notification ---")
notif1 = Notification(user, "Payment of \$19.99 received successfully.", "Payment")
print_result(notif1.get_type() == "Payment", "Type is Payment")
print_result(not notif1.is_read(), "Initially unread")
print_result(len(user.get_notifications()) == 1, "Notification added to user")
print(f" {notif1}")
notif1.mark_as_read()
print_result(notif1.is_read(), "Marked as read successfully")
print(f" After read: {notif1}")

Example 2: Subscription and Content notifications
print("\n --- Example 2: Subscription change & content update ---")
notif2 = Notification(user, "Your subscription has been upgraded to Premium!", "Subscription")
notif3 = Notification(user, "New release: 'Dune: Part Three' is now available!", "Content")
print_result(len(user.get_notifications()) == 3, "User has 3 notifications")
print_result(notif2.get_type() == "Subscription", "Subscription notification type correct")
print_result(notif3.get_type() == "Content", "Content notification type correct")
print(f" {notif2}")
print(f" {notif3}")
Exception handling
print("\n --- Exception Handling ---")
try:
bad_notif = Notification(user, "", "General")
print_result(False, "Empty message should raise ValueError")
except ValueError as e:
print_result(True, f"Empty message rejected: {e}")
try:
bad_notif = Notification(user, "Test", "InvalidType")
print_result(False, "Invalid type should raise ValueError")
except ValueError as e:
print_result(True, f"Invalid type rejected: {e}")
=====
RUN ALL TESTS
=====
def run_all_tests():
"""Execute all test cases and display results."""
print("=" * 60)
print(" STREAMFLIX – COMPREHENSIVE TEST SUITE")
print("=" * 60)
test_user_account_creation()
test_content_catalogue_management()
test_subscription_plan_management()
test_streaming_content_access()
test_payment_processing()
test_invoice_generation()
test_viewing_history_tracking()
test_notification_system()
print("\n" + "=" * 60)
print(" ALL TESTS COMPLETED SUCCESSFULLY")
print("=" * 60)
if __name__ == "__main__":
run_all_tests()

[Copy-paste the complete code from all files: [content.py](#), [user.py](#), [subscription.py](#), [payment.py](#), [viewing_history.py](#), [notification.py](#), [streaming_service.py](#), [main.py](#), [test_streamflix.py](#) — OR the single-file Colab version as provided above]

Section C: Repository Link

Google Colab link: <https://github.com/>

Section D: Summary of Learnings

Through this assignment, I gained practical experience in several key areas of object-oriented programming and software design.

First, I learned how to translate real-world requirements into a formal UML class diagram, identifying entities, their attributes, and the relationships between them. Understanding the distinctions between inheritance, composition, and aggregation was crucial — for example, modelling Admin as a subclass of User (IS-A), ContentCatalogue owning Content objects (composition), and Subscription referencing a shared SubscriptionPlan (aggregation).

Second, implementing the UML design in Python reinforced the importance of encapsulation through private attributes (name-mangling with double underscores) and public getter/setter methods. This ensures data integrity and prevents external code from modifying internal state directly.

Third, writing comprehensive test cases with both valid and invalid inputs taught me the importance of exception handling. Using try/except blocks to validate user inputs (empty names, invalid emails, negative prices, out-of-range ratings) ensures the system is robust and provides meaningful error messages.

Fourth, modular file organisation (separating each class into its own file) improved code readability and maintainability. Each module has a single responsibility, making it easier to locate, modify, and test individual components.

Finally, designing the streaming access control logic in StreamingService demonstrated how business rules (subscription tiers, premium content restrictions, availability checks) can be cleanly encoded in method logic, producing clear and informative user-facing messages for both successful and denied access attempts.